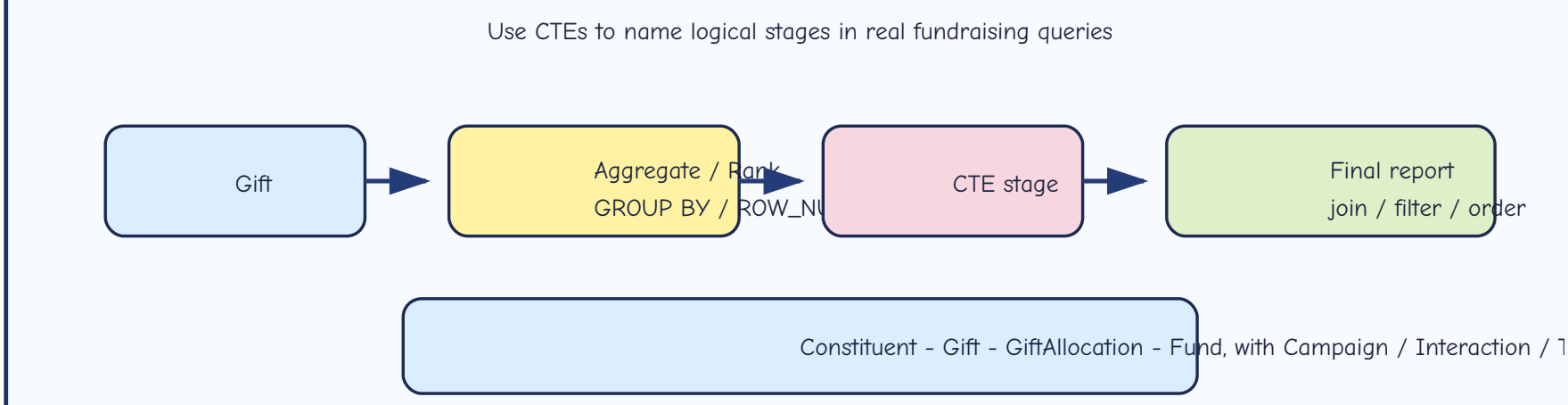
REVISION CHEAT SHEET:

SQL Server Common Table Expressions - CTEs

INTERVIEW-READY NOTES - RANKING - CTEs - RECURSION - SQL - DIAGRAMS

DIAGRAM

FundraisingDb CTE Learning Map



Topic & Subtopic Index

Study order: ranking functions -> CTE fundamentals -> joins and aggregation -> multiple CTEs -> recursive CTEs -> comparisons -> updates and performance -> mistakes -> real-world examples -> interview revision.

1. Ranking Functions

- Before CTEs - Ranking Functions
- ROW_NUMBER()
- PARTITION BY with ROW_NUMBER
- RANK()
- DENSE_RANK()
- ROW_NUMBER vs RANK vs DENSE_RANK
- NTILE()
- Ranking Function Quick Rule

2. CTE Fundamentals

- What Is a CTE?
- Simple CTE Example - Large Gifts
- Basic CTE Syntax
- Using ;WITH
- Named CTE Columns
- CTE Scope
- CTE Lifetime
- What Can Follow a CTE?
- Is a CTE a Temporary Table?
- CTE Flow Diagram

3. Filtering, Joins, and Aggregation

- CTE with WHERE
- CTE with an Outer WHERE
- CTE with JOIN
- CTE with Multiple JOINs
- CTE with GROUP BY
- CTE with GROUP BY and JOIN
- CTE with HAVING
- WHERE vs HAVING Inside CTE

4. Multiple CTEs and Ranking Patterns

- Multiple CTEs
- Multiple CTE Fundraising Example
- Practical CTE + ROW_NUMBER Pattern - Largest Gift per Constituent
- Latest Gift Per Constituent
- Top 3 Gifts Per Constituent
- Rank Campaigns by Total Giving
- Rank Funds by Allocated Amount

5. Practical Fundraising Patterns

- CTE for Conditional Aggregation
- CTE with GiftAllocation Validation
- CTE with Interaction Data
- Latest Interaction with Constituent and Fundraiser Names
- CTE with Task Data

6. Recursive CTEs

- Recursive CTE
- Recursive CTE Structure
- Anchor Member
- Recursive Member
- Campaign Month Recursive CTE
- Recursive Flow
- Recursive Termination
- MAXRECURSION
- Recursive CTE Data-Type Rule
- Recursive CTE Restrictions

7. Choosing CTE vs Other Tools

- CTE vs Subquery
- CTE vs Subquery Comparison
- CTE vs Temporary Table
- CTE vs #Temp Table
- CTE vs View
- CTE vs View Comparison

8. Updates and Performance

- Updating Data Through a CTE
- UPDATE Through CTE - Task Example
- DELETE Through CTE
- Is Every CTE Updateable?
- CTE Performance - Main Rule
- CTE Does Not Automatically Materialize
- CTE Does Not Replace Indexes
- Ranking Function Performance

9. Best Practices and Mistakes

- CTE with SELECT * - Best Practice
- Filter Early When Logically Correct
- Avoid Non-SARGable Date Filtering
- CTE Common Mistakes

10. Real-World Examples and Interview Review

- Complete Real-World CTE Example
- Another Practical Example - Campaign Performance
- CTE Decision Guide
- CTE Master Cheat Sheet
- Ranking Master Cheat Sheet
- 20 Detailed Interview Questions and Query Problems
- Final Revision Map
- Final Easy Memory Rule

A **Common Table Expression**, usually called a **CTE**, is a named query expression that exists for the SQL statement immediately following it.

CTEs are useful when a query becomes complex and we want to divide it into understandable logical steps.

A CTE does not create a permanent table.

It does not normally create a reusable temporary table.

Think of it as:

EXAMPLE / FLOW

```
A query
   ↓
Give that query a temporary name
   ↓
Use that name in the next SQL statement
```

Throughout this chapter, all practical examples use the existing `FundraisingDb` model:

EXAMPLE / FLOW

```
core.Tenant
core.AppUser

constituent.Constituent
constituent.Address
constituent.ConstituentAddress

fundraising.Campaign
fundraising.Fund
fundraising.Gift
fundraising.GiftAllocation

engagement.Interaction
engagement.Task
```

Those are the main business tables in your fundraising OLTP database.

The central fundraising flow is:

EXAMPLE / FLOW

```
Constituent
     |
     | gives
     v
Gift
     |
     +----------> Campaign
     |
     v
GiftAllocation
     |
     v
Fund
```

The schema defines Gift as the donation, Campaign as why money is raised, and GiftAllocation/Fund as where the gift money goes.

Before CTEs - Ranking Functions

Ranking functions are not CTEs.

However, ranking functions and CTEs are very commonly used together.

The most important ranking functions are:

EXAMPLE / FLOW

```
ROW_NUMBER()

RANK()

DENSE_RANK()

NTILE()
```

SQL Server currently provides these four ranking functions.

The three you should understand extremely well are:

EXAMPLE / FLOW

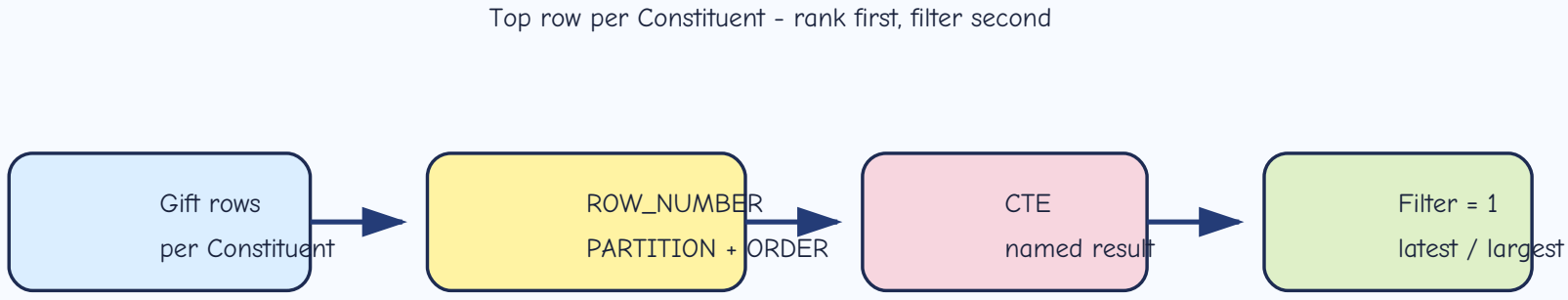
```
ROW_NUMBER
RANK
DENSE_RANK
```

Why Ranking Functions and CTEs Are Often Used Together

Suppose we need:

DIAGRAM

Ranking + CTE Pattern



Find the largest Gift made by each Constituent.

We can calculate a ranking:

EXAMPLE / FLOW

```
John's Gifts:

$1,000 → 1
$500   → 2
$250   → 3
$100   → 4
```

Then we want:

EXAMPLE / FLOW

```
only ranking = 1
```

A CTE makes this pattern easy:

EXAMPLE / FLOW

```
Gift rows
    ↓
Calculate ROW_NUMBER
    ↓
CTE
    ↓
Filter RowNumber = 1
```

This pattern appears constantly in real SQL development.

ROW_NUMBER()

Simple Definition

`ROW_NUMBER()` gives every row a unique sequential number according to the specified ordering.

Example:

Suppose John Smith has these Gifts:

GiftId	TotalAmount
10000	1000
10001	500
10002	500
10003	100

Using:

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    ORDER BY TotalAmount DESC
)
```

the result may be:

EXAMPLE / FLOW

```
$1000 → 1
$500  → 2
$500  → 3
$100  → 4
```

Even though two Gifts have the same amount:

EXAMPLE / FLOW

```
$500
$500
```

they still receive different row numbers.

ROW_NUMBER Syntax

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    ORDER BY expression
)
```

Fundraising example:

EXAMPLE / FLOW

```
SELECT
    GiftId,
    ConstituentId,
    TotalAmount,

    ROW_NUMBER()
    OVER
    (
        ORDER BY TotalAmount DESC,
                 GiftId
    ) AS RowNumber

FROM fundraising.Gift;
```

Why Add GiftId to ORDER BY?

Suppose two Gifts have the same:

EXAMPLE / FLOW

```
TotalAmount = 500
```

If we use only:

EXAMPLE / FLOW

```
ORDER BY TotalAmount DESC
```

SQL Server does not have a unique tie-breaker.

For predictable numbering, use:

EXAMPLE / FLOW

```
ORDER BY
    TotalAmount DESC,
    GiftId
```

Now every row has a deterministic tie-breaker.

Ranking functions are nondeterministic when the ordering does not uniquely determine row order.

PARTITION BY with ROW_NUMBER

`PARTITION BY` restarts the numbering for each group.

Suppose we want to rank Gifts separately for each Constituent.

EXAMPLE / FLOW

```
SELECT
    GiftId,
    ConstituentId,
    TotalAmount,

    ROW_NUMBER()
    OVER
    (
        PARTITION BY ConstituentId
        ORDER BY TotalAmount DESC,
                 GiftId
    ) AS GiftRank

FROM fundraising.Gift;
```

Conceptually:

EXAMPLE / FLOW

```
John Smith
------------
$1000 → 1
$500  → 2
$100  → 3

Jane Doe
------------
$2000 → 1
$750  → 2
$250  → 3
```

The numbering restarts for each:

EXAMPLE / FLOW

```
ConstituentId
```

ROW_NUMBER with CTE

Now suppose the requirement is:



Return only the largest Gift for each Constituent.

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY TotalAmount DESC,
                     GiftId
        ) AS GiftRank

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM RankedGifts
WHERE GiftRank = 1;
```

Understand the Query Step by Step

Step 1

Read all Gift rows:

EXAMPLE / FLOW

```
fundraising.Gift
```

Step 2

Divide Gifts by:

EXAMPLE / FLOW

```
ConstituentId
```

Step 3

Within each Constituent, sort by:

EXAMPLE / FLOW

```
TotalAmount DESC
```

Step 4

Number them:

EXAMPLE / FLOW

```
1
2
3
...
```

Step 5

The outer query keeps:

EXAMPLE / FLOW

```
GiftRank = 1
```

Result:

One largest Gift for each Constituent.

RANK() Simple Definition

`RANK()` assigns the same rank to tied values.

After a tie, it leaves a gap.

Suppose Gift amounts are:

EXAMPLE / FLOW

```
$1,000
$500
$500
$100
```

`RANK()` produces:

EXAMPLE / FLOW

```
$1,000 → 1
$500   → 2
$500   → 2
$100   → 4
```

Rank 3 is skipped.

RANK Fundraising Example

EXAMPLE / FLOW

```
SELECT
    GiftId,
    TotalAmount,

    RANK()
    OVER
    (
        ORDER BY TotalAmount DESC
    ) AS GiftRank

FROM fundraising.Gift;
```

Use `RANK()` when ties should represent the same position.

Example business question:

> Rank Gifts by donation amount, and if two Gifts have the same amount, give them the same ranking position.

DENSE_RANK()

Simple Definition

`DENSE_RANK()` also gives the same rank to tied values.

But it does **not** leave gaps.

Using:

EXAMPLE / FLOW

```
$1,000
$500
$500
$100
```

`DENSE_RANK()` returns:

EXAMPLE / FLOW

```
$1,000 → 1
$500   → 2
$500   → 2
$100   → 3
```

DENSE_RANK Fundraising Example

EXAMPLE / FLOW

```
SELECT
    GiftId,
    TotalAmount,

    DENSE_RANK()
    OVER
    (
        ORDER BY TotalAmount DESC
    ) AS GiftRank

FROM fundraising.Gift;
```

ROW_NUMBER vs RANK vs DENSE_RANK

Suppose:

EXAMPLE / FLOW

```
Gift Amounts

$1,000
$500
$500
$100
```

Amount	ROW_NUMBER	RANK	DENSE_RANK
1000	1	1	1
500	2	2	2
500	3	2	2
100	4	4	3

Easy memory:

EXAMPLE / FLOW

```
ROW_NUMBER
→ Every row gets a different number.

RANK
→ Ties share rank.
→ Gaps occur.

DENSE_RANK
→ Ties share rank.
→ No gaps.
```

NTILE()

`NTILE()` divides ordered rows into a specified number of groups.

Example:

Divide Constituents' Gifts into four donation-value groups.

EXAMPLE / FLOW

```
SELECT
    GiftId,
    ConstituentId,
    TotalAmount,

    NTILE(4)
    OVER
    (
        ORDER BY TotalAmount DESC
    ) AS GivingQuartile

FROM fundraising.Gift;
```

Possible meaning:

EXAMPLE / FLOW

```
1 → Highest donation group
2 → Next group
3 → Next group
4 → Lowest group
```

Use it for:

EXAMPLE / FLOW

```
quartiles

segments

buckets

reporting groups
```

Ranking Function Quick Rule

Use:

EXAMPLE / FLOW

```
ROW_NUMBER
```

when you need:

EXAMPLE / FLOW

```
exactly one first row
```

Use:

EXAMPLE / FLOW

```
RANK
```

when competition-style ranking matters.

Use:

EXAMPLE / FLOW

```
DENSE_RANK
```

when ties should share rank but rank numbers should remain continuous.

Use:

EXAMPLE / FLOW

```
NTILE
```

when dividing rows into a fixed number of ordered groups.

What Is a CTE?

Simple Definition

A **CTE is a named query expression that can be referenced by the statement immediately following it.**

Basic structure:

EXAMPLE / FLOW

```
WITH CTEName AS
(
    SELECT ...
)

SELECT ...
FROM CTEName;
```

Think:

EXAMPLE / FLOW

```
SELECT query
      ↓
Give it a name
      ↓
Use that name
```

Simple CTE Example - Large Gifts

Suppose the business requirement is:

Show Gifts greater than or equal to $1,000.

Without CTE:

EXAMPLE / FLOW

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

With CTE:

EXAMPLE / FLOW

```
WITH LargeGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
)

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM LargeGifts;
```

Does This Simple Query Need a CTE?

No.

The normal query is already easy.

This example is only to understand syntax.

Use CTEs when they improve:

EXAMPLE / FLOW

```
readability

organization

multi-step logic

ranking

aggregation

recursion
```

Do not add a CTE merely because you can.

Basic CTE Syntax

EXAMPLE / FLOW

```
WITH CTEName AS
(
    SELECT
        ...
    FROM ...
    WHERE ...
)

SELECT
    ...
FROM CTEName;
```

Using ;WITH

You will often see:

EXAMPLE / FLOW

```
;WITH LargeGifts AS
(
    ...
)
SELECT ...
```

Why the semicolon?

The statement before a CTE in the same batch must be properly terminated.

Starting with:

EXAMPLE / FLOW

```
;WITH
```

is a common defensive coding style.

Current SQL Server documentation requires the statement before the CTE to be terminated with a semicolon when another statement precedes it in the same batch.

Named CTE Columns

You may explicitly provide column names.

Example:

EXAMPLE / FLOW

```
WITH GiftSummary
(
    GiftId,
    DonorId,
    Amount
)
AS
(
    SELECT
        GiftId,
        ConstituentId,
        TotalAmount
    FROM fundraising.Gift
)

SELECT *
FROM GiftSummary;
```

Here:

EXAMPLE / FLOW

```
DonorId
```

is the CTE output name for:

EXAMPLE / FLOW

```
ConstituentId
```

The number of CTE column names must match the number of columns returned by the CTE query.

CTE Scope

A CTE does not exist permanently.

Example:

EXAMPLE / FLOW

```
WITH LargeGifts AS
(
    SELECT *
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
)

SELECT *
FROM LargeGifts;
```

After this statement finishes:

EXAMPLE / FLOW

```
SELECT *
FROM LargeGifts;
```

will not work.

The CTE no longer exists.

CTE Lifetime

Easy memory:

EXAMPLE / FLOW

```
CTE
=
One statement
```

It does not survive:

EXAMPLE / FLOW

```
another SELECT

GO

another batch

another session
```

What Can Follow a CTE?

A CTE can be used with statements such as:

EXAMPLE / FLOW

```
SELECT

INSERT

UPDATE

DELETE

MERGE
```

and can also participate in view definitions.

Is a CTE a Temporary Table?

No.

This is a very important distinction.

A CTE is:

EXAMPLE / FLOW

```
a named query expression
```

not:

EXAMPLE / FLOW

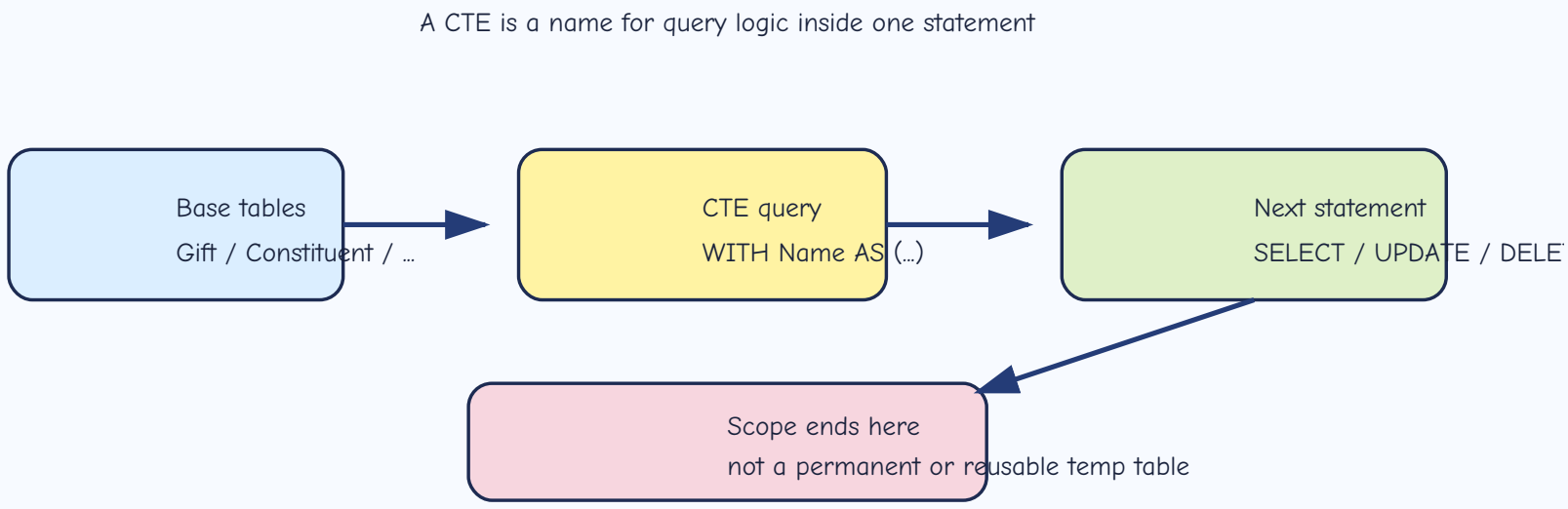
```
a physical #temp table
```

SQL Server does not normally materialize a CTE into a separately reusable object. Multiple references may cause the underlying CTE logic to be processed again.

CTE Flow Diagram

DIAGRAM

CTE Scope and Flow



A CTE is consumed by the immediately following statement, then its scope ends.

EXAMPLE / FLOW

```
fundraising.Gift
        |
        v
CTE query
        |
        v
Named logical result
        |
        v
Outer SELECT / UPDATE / DELETE
```

CTE with WHERE

Suppose:

Find Gifts made during 2026.

EXAMPLE / FLOW

```
WITH Gifts2026 AS
(
    SELECT
        GiftId,
        ConstituentId,
        CampaignId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE GiftDate >= '2026-01-01'
      AND GiftDate <  '2027-01-01'
)

SELECT *
FROM Gifts2026;
```

The filtering occurs inside the CTE.

CTE with an Outer WHERE

You may also filter after the CTE.

EXAMPLE / FLOW

```
WITH GiftData AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
)

SELECT *
FROM GiftData
WHERE TotalAmount >= 5000;
```

Which WHERE Should Be Used?

Place the condition where it logically belongs.

If the CTE is intended to mean:

EXAMPLE / FLOW

```
2026 Gifts
```

then:

EXAMPLE / FLOW

```
WHERE GiftDate >= '2026-01-01'
  AND GiftDate <  '2027-01-01'
```

belongs naturally inside that CTE.

If the CTE represents all Gift data and only one outer query needs a particular condition, putting it outside may make more sense.

CTE with JOIN

Suppose we need:

> Show Gift information with the Constituent who donated it.

Your schema defines:

EXAMPLE / FLOW

```
Constituent 1:M Gift
```

and `Gift.ConstituentId` references the Constituent.

EXAMPLE / FLOW

```
WITH GiftDonorDetails AS
(
    SELECT
        g.GiftId,
        g.GiftDate,
        g.TotalAmount,
        c.ConstituentId,
        c.DisplayName
    FROM fundraising.Gift AS g

    INNER JOIN constituent.Constituent AS c
        ON c.ConstituentId = g.ConstituentId
)

SELECT *
FROM GiftDonorDetails;
```

Understand This Example

The CTE does not replace the JOIN.

The JOIN still performs the relationship:

EXAMPLE / FLOW

```
Gift
+
Constituent
```

The CTE simply names that joined result:

EXAMPLE / FLOW

```
GiftDonorDetails
```

CTE with Multiple JOINs

Suppose the requirement is:

> Show the donor, Gift, Campaign, Fund, and allocated amount.

The business chain is:

EXAMPLE / FLOW

```
Constituent
    ↓
Gift
    ↓
GiftAllocation
    ↓
Fund

Gift
    ↓
Campaign
```

Example:

EXAMPLE / FLOW

```
WITH GiftAllocationDetails AS
(
    SELECT
        g.GiftId,
        g.GiftDate,
        g.TotalAmount,

        c.ConstituentId,
        c.DisplayName AS ConstituentName,

        ca.CampaignId,
        ca.CampaignName,

        f.FundId,
        f.FundName,

        ga.AllocationAmount

    FROM fundraising.Gift AS g
```

EXAMPLE / FLOW (CONT. 2)

```
    INNER JOIN constituent.Constituent AS c
        ON c.ConstituentId = g.ConstituentId

    LEFT JOIN fundraising.Campaign AS ca
        ON ca.CampaignId = g.CampaignId

    INNER JOIN fundraising.GiftAllocation AS ga
        ON ga.GiftId = g.GiftId

    INNER JOIN fundraising.Fund AS f
        ON f.FundId = ga.FundId
)

SELECT *
FROM GiftAllocationDetails;
```

What One Row Represents Here

This is critical.

One row represents:

> One Gift allocation to one Fund, together with its related donor and campaign information.

Suppose:

EXAMPLE / FLOW

```
Gift 10000
TotalAmount = $1,000
```

is allocated:

EXAMPLE / FLOW

```
$700 → Cancer Research Fund

$300 → Patient Assistance Fund
```

Then the CTE returns two rows for Gift 10000.

That is correct because there are two `GiftAllocation` rows.

CTE with GROUP BY

Suppose:

> Calculate total lifetime giving per Constituent.

EXAMPLE / FLOW

```
WITH ConstituentGiving AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving,
        AVG(TotalAmount) AS AverageGift,
        MAX(TotalAmount) AS LargestGift

    FROM fundraising.Gift

    GROUP BY ConstituentId
)

SELECT *
FROM ConstituentGiving;
```

Understand the Aggregation

Before aggregation:

EXAMPLE / FLOW

```
John Smith
$100
$500
$250
$1,000
```

After `GROUP BY ConstituentId`:

EXAMPLE / FLOW

```
John Smith

GiftCount      = 4
LifetimeGiving = $1,850
AverageGift    = $462.50
LargestGift    = $1,000
```

The CTE represents:

EXAMPLE / FLOW

```
one summary row per Constituent
```

CTE with GROUP BY and JOIN

Now let's include the Constituent's name.

EXAMPLE / FLOW

```
WITH ConstituentGiving AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving,
        MAX(TotalAmount) AS LargestGift

    FROM fundraising.Gift

    GROUP BY ConstituentId
)

SELECT
    c.ConstituentId,
    c.DisplayName,
    cg.GiftCount,
    cg.LifetimeGiving,
    cg.LargestGift
```

EXAMPLE / FLOW (CONT. 2)

```
FROM ConstituentGiving AS cg

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = cg.ConstituentId;
```

Why Aggregate Before Joining Here?

The CTE first reduces:

EXAMPLE / FLOW

```
many Gift rows
```

into:

EXAMPLE / FLOW

```
one row per Constituent
```

Then we join that summary to `Constituent`.

This often makes the purpose easier to understand.

Multiple CTEs

You can define more than one CTE in the same statement.

Syntax:

EXAMPLE / FLOW

```
WITH CTE1 AS
(
    ...
),
CTE2 AS
(
    ...
),
CTE3 AS
(
    ...
)

SELECT ...
```

Use:

EXAMPLE / FLOW

```
one WITH
```

and separate CTEs using commas.

Multiple CTE Fundraising Example

Requirement:



Calculate donor lifetime giving and rank Constituents from highest lifetime giving to lowest.

EXAMPLE / FLOW

```
WITH ConstituentTotals AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
),
RankedConstituents AS
(
    SELECT
        ConstituentId,
        GiftCount,
        LifetimeGiving,
```

EXAMPLE / FLOW (CONT. 2)

```
        DENSE_RANK()
        OVER
        (
            ORDER BY LifetimeGiving DESC
        ) AS GivingRank

    FROM ConstituentTotals
)

SELECT
    c.ConstituentId,
    c.DisplayName,
    r.GiftCount,
    r.LifetimeGiving,
    r.GivingRank

FROM RankedConstituents AS r
```

EXAMPLE / FLOW (CONT. 3)

```
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = r.ConstituentId

ORDER BY
    r.GivingRank,
    c.ConstituentId;
```

Understand the Multiple CTE Flow

EXAMPLE / FLOW

```
fundraising.Gift
       |
       v
ConstituentTotals
       |
       | SUM gifts
       v
One row per Constituent
       |
       v
RankedConstituents
       |
       | DENSE_RANK
       v
Ranked donors
       |
       v
JOIN Constituent
       |
       v
```

EXAMPLE / FLOW (CONT. 2)

```
Final report
```

Can One CTE Reference Another?

Yes.

A later CTE can reference an earlier CTE.

Here:

EXAMPLE / FLOW

```
RankedConstituents
```

references:

EXAMPLE / FLOW

```
ConstituentTotals
```

Forward References

This is not allowed:

EXAMPLE / FLOW

```
CTE1 uses CTE2

but CTE2 is declared later
```

Define dependencies in logical order:

EXAMPLE / FLOW

```
CTE1
   ↓
CTE2
   ↓
CTE3
```

CTE with HAVING

Suppose:

> Find Constituents whose lifetime giving is at least $10,000.

EXAMPLE / FLOW

```
WITH MajorDonors AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId

    HAVING SUM(TotalAmount) >= 10000
)

SELECT
    c.ConstituentId,
    c.DisplayName,
    m.LifetimeGiving

FROM MajorDonors AS m
```

EXAMPLE / FLOW (CONT. 2)

```
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = m.ConstituentId;
```

WHERE vs HAVING Inside CTE

Remember:

EXAMPLE / FLOW

```
WHERE
→ Filters Gift rows before grouping.

HAVING
→ Filters groups after aggregation.
```

Example:

EXAMPLE / FLOW

```
WHERE GiftDate >= '2026-01-01'
  AND GiftDate <  '2027-01-01'
```

limits Gifts participating in aggregation.

Then:

EXAMPLE / FLOW

```
HAVING SUM(TotalAmount) >= 10000
```

keeps only Constituents whose 2026 total reaches $10,000.

Combined Example

EXAMPLE / FLOW

```
WITH MajorDonors2026 AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS TotalGiving

    FROM fundraising.Gift

    WHERE GiftDate >= '2026-01-01'
      AND GiftDate <  '2027-01-01'

    GROUP BY ConstituentId

    HAVING SUM(TotalAmount) >= 10000
)

SELECT *
FROM MajorDonors2026;
```

CTE with ORDER BY

This rule is important.

An ordinary CTE cannot normally contain a final `ORDER BY` like this:

EXAMPLE / FLOW

```
WITH GiftData AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift

    ORDER BY TotalAmount DESC
)

SELECT *
FROM GiftData;
```

That is not valid as a normal CTE ordering request.

Ordinary `ORDER BY` inside a CTE definition is restricted unless used in an allowed context such as `TOP` or `OFFSET/FETCH`.

Correct Way to Sort Final CTE Output

EXAMPLE / FLOW

```
WITH GiftData AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift
)

SELECT *
FROM GiftData

ORDER BY
    TotalAmount DESC,
    GiftId;
```

Window Function ORDER BY Is Different

This is valid:

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            ORDER BY TotalAmount DESC
        ) AS GiftRank

    FROM fundraising.Gift
)

SELECT *
FROM RankedGifts;
```

Why?

Because:

EXAMPLE / FLOW

```
ORDER BY TotalAmount DESC
```

belongs to:

EXAMPLE / FLOW

```
ROW_NUMBER() OVER(...)
```

It controls ranking.

It does **not** promise final result ordering.

If final output order matters, still write:

EXAMPLE / FLOW

```
ORDER BY GiftRank;
```

in the outer query.

Practical CTE + ROW_NUMBER Pattern - Largest Gift per Constituent

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY TotalAmount DESC,
                     GiftId
        ) AS GiftRank

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM RankedGifts

WHERE GiftRank = 1;
```

This is one of the most useful CTE patterns.

Latest Gift Per Constituent

Now change the ordering.

Requirement:

Find each Constituent's most recent Gift.

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY GiftDate DESC,
                     GiftId DESC
        ) AS RowNumber

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM RankedGifts

WHERE RowNumber = 1;
```

Why GiftId DESC Is Included

Suppose a Constituent makes two Gifts on the same date.

EXAMPLE / FLOW

```
GiftDate = 2026-09-19
GiftDate = 2026-09-19
```

`GiftId DESC` provides a stable tie-breaker.

Top 3 Gifts Per Constituent

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY TotalAmount DESC,
                     GiftId
        ) AS GiftPosition

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount,
    GiftPosition

FROM RankedGifts

WHERE GiftPosition <= 3

ORDER BY
    ConstituentId,
    GiftPosition;
```

This gives:

EXAMPLE / FLOW

```
Top 3 Gifts
for each Constituent.
```

Rank Campaigns by Total Giving

Requirement:

> Which campaigns have raised the most money?

First aggregate Gifts by Campaign.

Then rank those Campaign totals.

EXAMPLE / FLOW

```
WITH CampaignTotals AS
(
    SELECT
        CampaignId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS AmountRaised

    FROM fundraising.Gift

    WHERE CampaignId IS NOT NULL

    GROUP BY CampaignId
),
RankedCampaigns AS
(
    SELECT
        CampaignId,
        GiftCount,
        AmountRaised,
```

EXAMPLE / FLOW (CONT. 2)

```
        DENSE_RANK()
        OVER
        (
            ORDER BY AmountRaised DESC
        ) AS CampaignRank

    FROM CampaignTotals
)

SELECT
    c.CampaignId,
    c.CampaignName,
    r.GiftCount,
    r.AmountRaised,
    r.CampaignRank

FROM RankedCampaigns AS r
```

EXAMPLE / FLOW (CONT. 3)

```
INNER JOIN fundraising.Campaign AS c
    ON c.CampaignId = r.CampaignId

ORDER BY
    r.CampaignRank,
    c.CampaignId;
```

Rank Funds by Allocated Amount

Your `GiftAllocation` table contains:

EXAMPLE / FLOW

```
GiftId
FundId
AllocationAmount
```

and connects Gifts to Funds.

EXAMPLE / FLOW

```
WITH FundTotals AS
(
    SELECT
        FundId,
        COUNT(DISTINCT GiftId) AS GiftCount,
        SUM(AllocationAmount) AS TotalAllocated

    FROM fundraising.GiftAllocation

    GROUP BY FundId
),
RankedFunds AS
(
    SELECT
        FundId,
        GiftCount,
        TotalAllocated,
```

EXAMPLE / FLOW (CONT. 2)

```
        RANK()
        OVER
        (
            ORDER BY TotalAllocated DESC
        ) AS FundRank

    FROM FundTotals
)

SELECT
    f.FundId,
    f.FundName,
    r.GiftCount,
    r.TotalAllocated,
    r.FundRank

FROM RankedFunds AS r
```

EXAMPLE / FLOW (CONT. 3)

```
INNER JOIN fundraising.Fund AS f
    ON f.FundId = r.FundId

ORDER BY
    r.FundRank,
    f.FundId;
```

CTE for Conditional Aggregation

Suppose we want a Constituent report showing:

EXAMPLE / FLOW

```
Total gifts

Large gifts ≥ $1,000

Very large gifts ≥ $10,000
```

EXAMPLE / FLOW

```
WITH ConstituentGiving AS
(
    SELECT
        ConstituentId,

        COUNT(*) AS GiftCount,

        SUM(TotalAmount) AS TotalGiving,

        SUM
        (
            CASE
                WHEN TotalAmount >= 1000
                    THEN 1
                ELSE 0
            END
        ) AS LargeGiftCount,
```

EXAMPLE / FLOW (CONT. 2)

```
        SUM
        (
            CASE
                WHEN TotalAmount >= 10000
                    THEN 1
                ELSE 0
            END
        ) AS MajorGiftCount

    FROM fundraising.Gift

    GROUP BY ConstituentId
)

SELECT *
FROM ConstituentGiving;
```

CTE with GiftAllocation Validation

Business rule:

> The sum of GiftAllocation amounts should normally match the Gift's TotalAmount.

We can use a CTE to calculate allocation totals.

EXAMPLE / FLOW

```
WITH AllocationTotals AS
(
    SELECT
        GiftId,
        SUM(AllocationAmount) AS AllocatedAmount

    FROM fundraising.GiftAllocation

    GROUP BY GiftId
)

SELECT
    g.GiftId,
    g.TotalAmount,
    a.AllocatedAmount,

    g.TotalAmount - a.AllocatedAmount
        AS Difference
```

EXAMPLE / FLOW (CONT. 2)

```
FROM fundraising.Gift AS g

INNER JOIN AllocationTotals AS a
    ON a.GiftId = g.GiftId

WHERE g.TotalAmount <> a.AllocatedAmount;
```

What Does This Query Find?

Suppose:

EXAMPLE / FLOW

```
Gift.TotalAmount
= $1,000
```

but allocations total:

EXAMPLE / FLOW

```
$700
+
$200
=
$900
```

Difference:

EXAMPLE / FLOW

```
$100
```

The query exposes the mismatch.

This is an excellent real-world CTE use:

EXAMPLE / FLOW

```
calculate something first
then compare it to another table.
```

CTE with Interaction Data

An `Interaction` records communication between an AppUser and Constituent. Your schema connects `Interaction` to `Constituent` and `AppUser` through `ConstituentId` and `FundraiserUserId`.

Requirement:



Find the latest Interaction for each Constituent.

EXAMPLE / FLOW

```
WITH RankedInteractions AS
(
    SELECT
        InteractionId,
        ConstituentId,
        FundraiserUserId,
        InteractionType,
        InteractionDate,
        Subject,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY InteractionDate DESC,
                     InteractionId DESC
        ) AS RowNumber
```

EXAMPLE / FLOW (CONT. 2)

```
    FROM engagement.Interaction
)

SELECT
    InteractionId,
    ConstituentId,
    FundraiserUserId,
    InteractionType,
    InteractionDate,
    Subject

FROM RankedInteractions

WHERE RowNumber = 1;
```

Latest Interaction with Constituent and Fundraiser Names

EXAMPLE / FLOW

```
WITH RankedInteractions AS
(
    SELECT
        InteractionId,
        ConstituentId,
        FundraiserUserId,
        InteractionType,
        InteractionDate,
        Subject,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY InteractionDate DESC,
                     InteractionId DESC
        ) AS RowNumber
```

EXAMPLE / FLOW (CONT. 2)

```
    FROM engagement.Interaction
)

SELECT
    c.DisplayName AS ConstituentName,
    u.DisplayName AS FundraiserName,
    ri.InteractionType,
    ri.InteractionDate,
    ri.Subject

FROM RankedInteractions AS ri

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = ri.ConstituentId

INNER JOIN core.AppUser AS u
    ON u.AppUserId = ri.FundraiserUserId

WHERE ri.RowNumber = 1;
```

CTE with Task Data

Your `Task` table references:

EXAMPLE / FLOW

```
ConstituentId

AssignedToUserId

CreatedByUserId
```

and `AppUser` plays two different roles: task creator and assigned user.

Requirement:

Count open Tasks assigned to each AppUser.

For this example, consider a Task open when:

EXAMPLE / FLOW

```
CompletedAtUtc IS NULL
```

EXAMPLE / FLOW

```
WITH OpenTaskCounts AS
(
    SELECT
        AssignedToUserId,
        COUNT(*) AS OpenTaskCount

    FROM engagement.Task

    WHERE CompletedAtUtc IS NULL

    GROUP BY AssignedToUserId
)

SELECT
    u.AppUserId,
    u.DisplayName,
    o.OpenTaskCount

FROM OpenTaskCounts AS o
```

EXAMPLE / FLOW (CONT. 2)

```
INNER JOIN core.AppUser AS u
    ON u.AppUserId = o.AssignedToUserId

ORDER BY
    o.OpenTaskCount DESC,
    u.AppUserId;
```

Recursive CTE

A **recursive CTE** is a CTE that references itself.

This allows SQL Server to repeatedly generate or traverse another level.

Common recursive use cases include:

EXAMPLE / FLOW

```
organizational hierarchies

category trees

folder hierarchies

bill of materials

date/month generation

number sequences
```

SQL Server recursive CTEs consist of an anchor member and a recursive member, and recursion ends when the recursive member produces no rows.

Important Note About Your FundraisingDb

Your current `FundraisingDb` does **not** contain a self-referencing hierarchy such as:

EXAMPLE / FLOW

```
Employee.ManagerId
```

or:

EXAMPLE / FLOW

```
Category.ParentCategoryId.
```

Therefore I will not invent an unrelated Employee or Category table.

Also remember:

EXAMPLE / FLOW

```
Constituent
   ↓
Gift
   ↓
GiftAllocation
   ↓
Fund
```

is a relational chain, but it is **not recursive**.

Each level is a different table.

A recursive hierarchy normally repeats the same type of relationship across levels.

For your current schema, a realistic recursive example is:

Generate every month between a Campaign's StartDate and EndDate.

Campaign contains `StartDate` and `EndDate` in your model.

Recursive CTE Structure

A recursive CTE usually contains:

EXAMPLE / FLOW

```
Anchor Member

UNION ALL

Recursive Member
```

Basic syntax:

EXAMPLE / FLOW

```
WITH CTEName AS
(
    -- Anchor
    SELECT ...

    UNION ALL

    -- Recursive member
    SELECT ...
    FROM CTEName
    WHERE ...
)

SELECT *
FROM CTEName;
```

Anchor Member

Simple Definition

The anchor member creates the starting row or rows.

Think:



Where does recursion begin?

For a Campaign monthly calendar:

EXAMPLE / FLOW

```
Start with the Campaign's first month.
```

Recursive Member

The recursive member references the CTE itself and creates the next level.

For a Campaign:

EXAMPLE / FLOW

```
January
   ↓
February
   ↓
March
   ↓
April
```

Each recursion adds:

EXAMPLE / FLOW

```
1 month
```

Campaign Month Recursive CTE

Suppose Campaign 200 runs from:

EXAMPLE / FLOW

```
January 2026
through
December 2026
```

We want one row for each month.

EXAMPLE / FLOW

```
WITH CampaignMonths AS
(
    -- Anchor member
    SELECT
        CampaignId,
        CampaignName,

        CAST
        (
            DATEFROMPARTS
            (
                YEAR(StartDate),
                MONTH(StartDate),
                1
            )
            AS DATE
        ) AS MonthStart,

        EndDate
```

EXAMPLE / FLOW (CONT. 2)

```
    FROM fundraising.Campaign

    WHERE CampaignId = 200
      AND EndDate IS NOT NULL

    UNION ALL

    -- Recursive member
    SELECT
        CampaignId,
        CampaignName,

        CAST
        (
            DATEADD
            (
                MONTH,
                1,
                MonthStart
            )
```

EXAMPLE / FLOW (CONT. 3)

```
            AS DATE
        ),

        EndDate

    FROM CampaignMonths

    WHERE DATEADD
          (
              MONTH,
              1,
              MonthStart
          )
          <=
          DATEFROMPARTS
          (
              YEAR(EndDate),
              MONTH(EndDate),
              1
          )
```

EXAMPLE / FLOW (CONT. 4)

```
)

SELECT
    CampaignId,
    CampaignName,
    MonthStart

FROM CampaignMonths

ORDER BY MonthStart

OPTION
(
    MAXRECURSION 100
);
```

Understand the Recursive Campaign Example

Assume:

EXAMPLE / FLOW

```
StartDate = 2026-01-15

EndDate = 2026-04-30
```

Anchor

Produces:

EXAMPLE / FLOW

```
2026-01-01
```

First Recursion

Adds one month:

EXAMPLE / FLOW

```
2026-02-01
```

Second Recursion

EXAMPLE / FLOW

```
2026-03-01
```

Third Recursion

EXAMPLE / FLOW

```
2026-04-01
```

Next Attempt

Would produce:

EXAMPLE / FLOW

```
2026-05-01
```

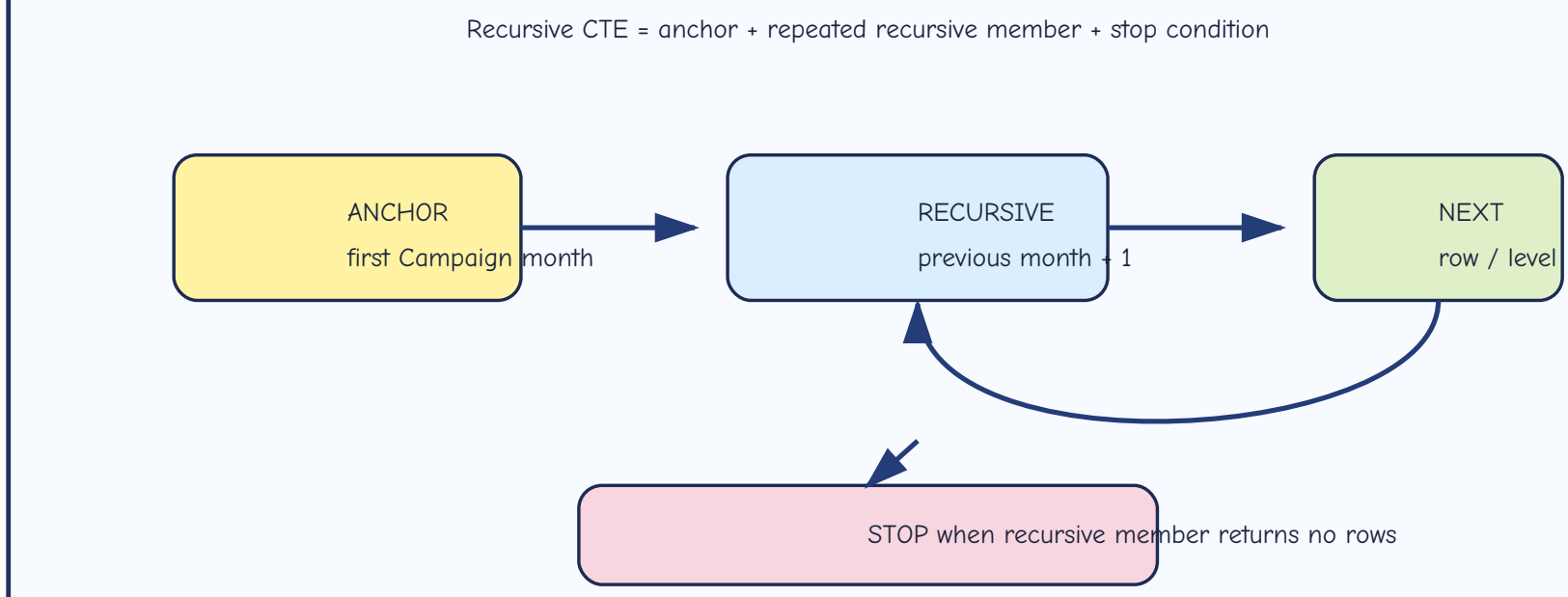
But that exceeds the EndDate's month.

Therefore recursion stops.

Recursive Flow

DIAGRAM

Recursive CTE Flow



MAXRECURSION is a safety limit; correct termination logic is still required.

EXAMPLE / FLOW

```
ANCHOR

January
   |
   v

RECURSION 1

February
   |
   v

RECURSION 2

March
   |
   v

RECURSION 3
```

EXAMPLE / FLOW (CONT. 2)

```
April
   |
   v

Next month outside campaign
   |
   v

STOP
```

Why UNION ALL?

SQL Server requires `UNION ALL` between the final anchor member and the first recursive member.

It combines:

EXAMPLE / FLOW

```
starting rows
+
rows generated at each recursive level
```

without performing duplicate removal during each step.

Recursive Termination

A recursive CTE must eventually stop.

For our Campaign example:

EXAMPLE / FLOW

```
WHERE DATEADD
(
    MONTH,
    1,
    MonthStart
)
<= DATEFROMPARTS
(
    YEAR(EndDate),
    MONTH(EndDate),
    1
)
```

is the terminating condition.

Once this condition becomes false:

EXAMPLE / FLOW

```
recursive member returns no rows
```

and recursion ends.

MAXRECURSION

SQL Server provides:

EXAMPLE / FLOW

```
OPTION
(
    MAXRECURSION n
)
```

The server default is:

EXAMPLE / FLOW

```
100
```

The allowed value range is:

EXAMPLE / FLOW

```
0 through 32767
```

where:

EXAMPLE / FLOW

```
0
```

means:

EXAMPLE / FLOW

```
no recursion limit.
```

Why MAXRECURSION Exists

Bad recursive logic can cause:

EXAMPLE / FLOW

```
infinite recursion
```

or unexpectedly large recursion.

Example conceptually:

EXAMPLE / FLOW

```
Current row
   ↓
Produces same current row
   ↓
Produces same row
   ↓
Forever
```

`MAXRECURSION` provides a safety boundary.

Should You Always Use MAXRECURSION 0?

No.

EXAMPLE / FLOW

```
OPTION
(
    MAXRECURSION 0
)
```

removes the limit.

Use this only when the recursion is known to terminate safely and the business requirement genuinely needs more than the normal limit.

Do not use it to hide an incorrect recursive query.

Recursive CTE Data-Type Rule

The anchor and recursive members must return:

EXAMPLE / FLOW

```
same number of columns
```

and corresponding recursive CTE columns need the same compatible SQL Server data types.

This is why recursive queries that build strings sometimes require explicit:

EXAMPLE / FLOW

```
CAST(...)
```

in both anchor and recursive members.

Recursive CTE Restrictions

The recursive member has additional SQL Server restrictions.

For example, operations such as:

EXAMPLE / FLOW

```
GROUP BY

HAVING

scalar aggregation

TOP

OUTER JOIN

subqueries
```

are restricted in the recursive member.

This is specifically about the recursive member, not normal nonrecursive CTEs.

Recursive CTE vs Normal CTE

Normal CTE

EXAMPLE / FLOW

```
Defined once
↓
Referenced by following statement
```

Example:

EXAMPLE / FLOW

```
Gift totals per Constituent
```

Recursive CTE

EXAMPLE / FLOW

```
Anchor
↓
CTE references itself
↓
Next level
↓
Next level
↓
Stop
```

Example:

EXAMPLE / FLOW

```
Campaign month generation
```

CTE vs Subquery

Suppose we want:



Latest Gift per Constituent.

Using a derived table:

EXAMPLE / FLOW

```
SELECT *
FROM
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY GiftDate DESC,
                     GiftId DESC
        ) AS RowNumber

    FROM fundraising.Gift
) AS RankedGifts
```

EXAMPLE / FLOW (CONT. 2)

```
WHERE RowNumber = 1;
```

Using CTE:

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY GiftDate DESC,
                     GiftId DESC
        ) AS RowNumber

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT *
FROM RankedGifts
WHERE RowNumber = 1;
```

CTE vs Subquery Comparison

CTE	Subquery / Derived Table
Named query expression	Inline query

CTE	Subquery / Derived Table
Often easier for multi-step SQL	Excellent for small logic
Multiple CTEs can form logical stages	Deep nesting may reduce readability
Supports recursion	Ordinary derived table doesn't self-recurse
Scope is one statement	Scope is containing query

When to Use CTE Instead of Subquery

Use a CTE when:

EXAMPLE / FLOW

```
logic has multiple stages

ranking is followed by filtering

several transformations need meaningful names

recursive processing is required

a nested query is becoming difficult to read
```

Use a simple subquery when:

EXAMPLE / FLOW

```
the logic is short,

used once,

and remains easy to understand.
```

CTE vs Temporary Table

A local temporary table looks like:

EXAMPLE / FLOW

```
CREATE TABLE #ConstituentGiving
(
    ConstituentId BIGINT,
    LifetimeGiving DECIMAL(18,2)
);
```

A temp table is an actual temporary database object.

A CTE is not.

CTE vs #Temp Table

CTE	Temp Table
One statement	Multiple statements

CTE	Temp Table
Named query expression	Temporary database object
Not independently indexed	Can create indexes
Not independently materialized for reuse	Data is materialized
Excellent for query organization	Excellent for staged processing
Supports recursion	Not recursively self-executing
Repeated references may repeat work	Can calculate once and reuse

CTE Performance Example

Suppose calculating donor lifetime giving is expensive.

CTE:

EXAMPLE / FLOW

```
WITH ConstituentTotals AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS LifetimeGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
)

SELECT ...
FROM ConstituentTotals AS a

JOIN ConstituentTotals AS b
    ON ...
```

If an expensive CTE result is referenced many times, the CTE is not automatically stored once as a reusable intermediate table.

A temp table can sometimes be more appropriate:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    SUM(TotalAmount) AS LifetimeGiving
INTO #ConstituentTotals
FROM fundraising.Gift
GROUP BY ConstituentId;
```

Then:

EXAMPLE / FLOW

```
CREATE INDEX IX_ConstituentTotals_ConstituentId
ON #ConstituentTotals(ConstituentId);
```

Now the materialized intermediate data can be reused.

Rule

Use:

EXAMPLE / FLOW

```
CTE
```

when:

EXAMPLE / FLOW

```
one statement needs understandable logical stages.
```

Consider:

EXAMPLE / FLOW

```
#Temp table
```

when:

EXAMPLE / FLOW

```
the intermediate data is expensive,

reused several times,

needed across statements,

or needs its own indexes/statistics.
```

CTE vs View

A CTE is temporary to one statement.

A view is a persistent database object.

Example:

EXAMPLE / FLOW

```
CREATE VIEW fundraising.vw_ConstituentGiving
AS
SELECT
    ConstituentId,
    COUNT(*) AS GiftCount,
    SUM(TotalAmount) AS LifetimeGiving
FROM fundraising.Gift
GROUP BY ConstituentId;
```

The view remains available until altered or dropped.

CTE vs View Comparison

CTE	View
One statement	Persistent
Defined inside query	Stored in database metadata
Good for query-specific logic	Good for reusable query abstraction
No separate permission object	View can have permissions

CTE	View
Can be recursive	View may contain a CTE, subject to SQL Server rules

Updating Data Through a CTE

A CTE can sometimes be the target of:

EXAMPLE / FLOW

```
UPDATE

DELETE
```

when SQL Server can map the CTE rows to updateable rows in an underlying table.

UPDATE Through CTE - Task Example

Suppose we want to raise the priority of overdue incomplete Tasks.

Your Task table contains:

EXAMPLE / FLOW

```
DueDate
CompletedAtUtc
Priority
```

according to the schema.

Example:

EXAMPLE / FLOW

```
WITH OverdueTasks AS
(
    SELECT
        TaskId,
        Priority,
        DueDate,
        CompletedAtUtc

    FROM engagement.Task

    WHERE DueDate < CAST(SYSUTCDATETIME() AS DATE)
      AND CompletedAtUtc IS NULL
)

UPDATE OverdueTasks
SET Priority = 'High';
```

This updates the matching rows in:

EXAMPLE / FLOW

```
engagement.Task
```

Important Safety Practice

Before an UPDATE through a CTE, run the same CTE with:

EXAMPLE / FLOW

```
SELECT *
FROM OverdueTasks;
```

first.

Verify exactly which rows will be changed.

Then execute the UPDATE.

DELETE Through CTE

You can also delete through a suitable CTE.

For learning, suppose you want to inspect duplicate or invalid staging-type rows.

The safe process is:

EXAMPLE / FLOW

```
1. Build CTE.

2. SELECT from CTE.

3. Verify rows.

4. Only then DELETE.
```

Do not immediately execute destructive SQL in production.

Is Every CTE Updateable?

No.

A simple CTE over one base table may often be updateable.

A CTE containing things such as:

EXAMPLE / FLOW

```
GROUP BY

aggregate summaries

DISTINCT

complex joins

derived rows
```

may not be directly updateable.

Example:

EXAMPLE / FLOW

```
WITH ConstituentTotals AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
)
```

`TotalGiving` is calculated from many rows.

There is no single base row that corresponds to that summary value.

So you cannot treat it like a simple updateable row.

CTE Performance - Main Rule

A CTE is not inherently:

EXAMPLE / FLOW

```
fast
```

or:

EXAMPLE / FLOW

```
slow.
```

A CTE mainly changes how we express the query.

Performance depends on:

EXAMPLE / FLOW

```
indexes

row counts

statistics

join conditions

filtering

sorting

aggregations

window functions

cardinality

number of CTE references
```

EXAMPLE / FLOW (CONT. 2)

```
recursion depth

execution plan
```

CTE Does Not Automatically Materialize

A very common misconception is:

> SQL Server runs the CTE once and stores it.

That is not guaranteed.

Microsoft states that CTE results are not automatically materialized as a reusable object; repeated references can cause the underlying query expression to be processed again.

CTE Does Not Replace Indexes

Suppose:

EXAMPLE / FLOW

```
WITH ConstituentGifts AS
(
    SELECT *
    FROM fundraising.Gift
    WHERE ConstituentId = 1001
)

SELECT *
FROM ConstituentGifts;
```

Using a CTE does not magically make:

EXAMPLE / FLOW

```
Gift.ConstituentId
```

fast.

Performance still depends heavily on useful indexing and the actual workload.

Important FundraisingDb Columns That Often Participate in Queries

Examples include:

EXAMPLE / FLOW

```
Gift.ConstituentId

Gift.CampaignId

GiftAllocation.GiftId

GiftAllocation.FundId

Interaction.ConstituentId

Interaction.FundraiserUserId

Task.ConstituentId

Task.AssignedToUserId
```

These relationships are central to your database model. Whether each specific index should exist depends on query workload and existing keys/indexes.

Ranking Function Performance

Consider:

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    PARTITION BY ConstituentId
    ORDER BY GiftDate DESC
)
```

SQL Server must logically organize rows according to:

EXAMPLE / FLOW

```
ConstituentId

GiftDate
```

Large ranking operations may involve expensive:

EXAMPLE / FLOW

```
sorting

memory grants

temporary work

spills to tempdb
```

when appropriate supporting order/indexes are unavailable.

CTE with SELECT * - Best Practice

Avoid unnecessary:

EXAMPLE / FLOW

```
SELECT *
```

in production-style CTEs.

Instead of:

EXAMPLE / FLOW

```
WITH GiftData AS
(
    SELECT *
    FROM fundraising.Gift
)
```

prefer:

EXAMPLE / FLOW

```
WITH GiftData AS
(
    SELECT
        GiftId,
        ConstituentId,
        CampaignId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
)
```

Benefits:

EXAMPLE / FLOW

```
clearer intent

less coupling

easier maintenance

only needed columns are exposed
```

Filter Early When Logically Correct

Suppose you only need 2026 Gifts.

Prefer expressing that requirement clearly:

EXAMPLE / FLOW

```
WITH Gifts2026 AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE GiftDate >= '2026-01-01'
      AND GiftDate <  '2027-01-01'
)

SELECT *
FROM Gifts2026;
```

Rather than unnecessarily building logic around all historical Gifts.

The optimizer can still transform predicates, but your SQL should clearly express the required data.

Avoid Non-SARGable Date Filtering

Less index-friendly pattern:

EXAMPLE / FLOW

```
WHERE YEAR(GiftDate) = 2026
```

Prefer:

EXAMPLE / FLOW

```
WHERE GiftDate >= '2026-01-01'
  AND GiftDate <  '2027-01-01'
```

The second form usually gives SQL Server a better opportunity to seek an appropriate index.

CTE Common Mistakes

Mistake 1 - Using CTE for Every Query

This:

EXAMPLE / FLOW

```
WITH GiftCTE AS
(
    SELECT *
    FROM fundraising.Gift
)

SELECT *
FROM GiftCTE;
```

adds little value.

Use CTEs when they improve the query.

Mistake 2 - Thinking CTE Persists

A CTE cannot be reused by several later independent SQL statements.

Use:

EXAMPLE / FLOW

```
#Temp table
```

when that is required.

Mistake 3 - Thinking CTE Is Automatically Stored

A normal CTE is not automatically a materialized result.

Mistake 4 - Incorrect ORDER BY

Wrong:

EXAMPLE / FLOW

```
WITH Gifts AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift

    ORDER BY TotalAmount DESC
)

SELECT *
FROM Gifts;
```

Sort the final result outside the CTE.

Mistake 5 - Forgetting Tie-Breakers

Less predictable:

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    ORDER BY TotalAmount DESC
)
```

Better when GiftId is unique:

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    ORDER BY TotalAmount DESC,
             GiftId
)
```

Mistake 6 - Using RANK When Exactly One Row Is Needed

Suppose two Gifts have the same largest amount.

EXAMPLE / FLOW

```
RANK()
```

may give both:

EXAMPLE / FLOW

```
Rank = 1
```

If the requirement is:



Return exactly one latest/largest row.

Use:

EXAMPLE / FLOW

```
ROW_NUMBER()
```

with an appropriate tie-breaker.

Mistake 7 - Confusing PARTITION BY with GROUP BY

GROUP BY:

EXAMPLE / FLOW

```
reduces multiple rows
into one summary row per group.
```

PARTITION BY:

EXAMPLE / FLOW

```
keeps the original rows
but lets a window function calculate separately per group.
```

Example:

EXAMPLE / FLOW

```
ROW_NUMBER()
OVER
(
    PARTITION BY ConstituentId
    ORDER BY GiftDate DESC
)
```

does not collapse Gift rows.

Every Gift remains.

Mistake 8 - Calling a Multi-Table Chain Recursive

This:

EXAMPLE / FLOW

```
Constituent
→ Gift
→ GiftAllocation
→ Fund
```

is not recursion.

It is a normal chain of joins.

Recursion involves repeatedly applying the same CTE relationship until no more rows are produced.

Mistake 9 - Removing MAXRECURSION Without Checking Logic

If a recursive query reaches 100 levels, investigate:

EXAMPLE / FLOW

```
Is the requirement legitimately deeper?

Is termination correct?

Is there a cycle?

Is the recursion logic wrong?
```

before using:

EXAMPLE / FLOW

```
MAXRECURSION 0
```

Mistake 10 - Reusing an Expensive CTE Repeatedly

If an expensive CTE is referenced multiple times, test whether a:

EXAMPLE / FLOW

```
#Temp table
```

would provide better performance.

Do not assume either approach is always faster.

Measure.

Complete Real-World CTE Example

Business requirement:

> For each Constituent, show the lifetime giving, Gift count, largest Gift, latest Gift date, and giving rank across all Constituents.

We can solve this in logical steps.

Step 1 - Aggregate Constituent Giving

EXAMPLE / FLOW

```
WITH ConstituentTotals AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving,
        MAX(TotalAmount) AS LargestGift,
        MAX(GiftDate) AS LatestGiftDate

    FROM fundraising.Gift

    GROUP BY ConstituentId
),
```

One output row now represents:

EXAMPLE / FLOW

one Constituent's giving summary.

Step 2 - Rank Constituents

EXAMPLE / FLOW

```
RankedConstituents AS
(
    SELECT
        ConstituentId,
        GiftCount,
        LifetimeGiving,
        LargestGift,
        LatestGiftDate,

        DENSE_RANK()
        OVER
        (
            ORDER BY LifetimeGiving DESC
        ) AS GivingRank

    FROM ConstituentTotals
)
```

Now each Constituent has a giving rank.

Step 3 - Join the Constituent Table

Full query:

EXAMPLE / FLOW

```
WITH ConstituentTotals AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving,
        MAX(TotalAmount) AS LargestGift,
        MAX(GiftDate) AS LatestGiftDate

    FROM fundraising.Gift

    GROUP BY ConstituentId
),
RankedConstituents AS
(
    SELECT
        ConstituentId,
        GiftCount,
        LifetimeGiving,
        LargestGift,
```

EXAMPLE / FLOW (CONT. 2)

```
        LatestGiftDate,

        DENSE_RANK()
        OVER
        (
            ORDER BY LifetimeGiving DESC
        ) AS GivingRank

    FROM ConstituentTotals
)

SELECT
    c.ConstituentId,
    c.DisplayName,
    r.GiftCount,
    r.LifetimeGiving,
    r.LargestGift,
    r.LatestGiftDate,
    r.GivingRank
```

EXAMPLE / FLOW (CONT. 3)

```
FROM RankedConstituents AS r

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = r.ConstituentId

ORDER BY
    r.GivingRank,
    c.ConstituentId;
```

Complete Flow

EXAMPLE / FLOW

```
Gift
 |
 | GROUP BY Constituent
 v
ConstituentTotals
 |
 | DENSE_RANK
 v
RankedConstituents
 |
 | JOIN
 v
Constituent
 |
 v
Final donor giving report
```

Another Practical Example - Campaign Performance

Business question:

For each Campaign, calculate the money raised, number of Gifts, number of unique Constituents, and rank Campaigns by total amount raised.

EXAMPLE / FLOW

```
WITH CampaignGiving AS
(
    SELECT
        CampaignId,

        COUNT(*) AS GiftCount,

        COUNT(DISTINCT ConstituentId)
            AS UniqueConstituentCount,

        SUM(TotalAmount)
            AS AmountRaised,

        AVG(TotalAmount)
            AS AverageGift

    FROM fundraising.Gift

    WHERE CampaignId IS NOT NULL
```

EXAMPLE / FLOW (CONT. 2)

```
    GROUP BY CampaignId
),
RankedCampaigns AS
(
    SELECT
        CampaignId,
        GiftCount,
        UniqueConstituentCount,
        AmountRaised,
        AverageGift,

        RANK()
        OVER
        (
            ORDER BY AmountRaised DESC
        ) AS CampaignRank

    FROM CampaignGiving
)
```

EXAMPLE / FLOW (CONT. 3)

```
SELECT
    c.CampaignId,
    c.CampaignName,
    c.GoalAmount,

    r.GiftCount,
    r.UniqueConstituentCount,
    r.AmountRaised,
    r.AverageGift,
    r.CampaignRank

FROM RankedCampaigns AS r

INNER JOIN fundraising.Campaign AS c
    ON c.CampaignId = r.CampaignId

ORDER BY
    r.CampaignRank,
    c.CampaignId;
```

CTE Decision Guide

Use a normal query when:

EXAMPLE / FLOW

```
the SQL is already simple.
```

Use a CTE when:

EXAMPLE / FLOW

```
one statement has several logical stages.
```

Use a CTE + ranking when:

EXAMPLE / FLOW

```
you need Top N,
latest row,
largest row,
or ranking per group.
```

Use a recursive CTE when:

EXAMPLE / FLOW

```
the same relationship/process
must repeat through multiple levels.
```

Use a subquery when:

EXAMPLE / FLOW

```
the nested logic is small and clear.
```

Use a temp table when:

EXAMPLE / FLOW

```
intermediate data is expensive,
reused repeatedly,
needed across statements,
or needs indexes.
```

Use a view when:

EXAMPLE / FLOW

```
the query abstraction should be
persistent and reusable.
```

CTE Master Cheat Sheet

EXAMPLE / FLOW

```
WITH Name AS
(
    SELECT ...
)
SELECT ...
FROM Name;
```

EXAMPLE / FLOW

```
CTE Scope
→ One following statement.
```

EXAMPLE / FLOW

```
Multiple CTEs
→ One WITH, comma-separated.
```

EXAMPLE / FLOW

```
CTE + JOIN
→ Organize related-table query.
```

EXAMPLE / FLOW

```
CTE + GROUP BY
→ Build summaries in stages.
```

EXAMPLE / FLOW

```
CTE + ROW_NUMBER
→ Latest/Top row per group.
```

EXAMPLE / FLOW

```
Recursive CTE
→ Anchor + UNION ALL + recursive member.
```

EXAMPLE / FLOW

```
MAXRECURSION
→ Controls recursive depth.
```

EXAMPLE / FLOW

```
CTE
≠ #Temp table.
```

EXAMPLE / FLOW

```
CTE
≠ automatic performance optimization.
```

Ranking Master Cheat Sheet

EXAMPLE / FLOW

```
ROW_NUMBER
1,2,3,4
Every row unique
```

EXAMPLE / FLOW

```
RANK
1,2,2,4
Tie + gap
```

EXAMPLE / FLOW

```
DENSE_RANK
1,2,2,3
Tie + no gap
```

EXAMPLE / FLOW

```
NTILE(4)
Split ordered rows into 4 groups
```

20 Detailed Interview Questions and Query Problems

Interview Question 1

What is a Common Table Expression?

A CTE is a named query expression defined using **WITH** and available to the SQL statement immediately following it.

It improves readability and is especially useful for:

EXAMPLE / FLOW

```
multi-step queries

aggregation

ranking

recursive processing
```

Example:

EXAMPLE / FLOW

```
WITH LargeGifts AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= 1000
)

SELECT *
FROM LargeGifts;
```

`LargeGifts` is not a permanent table.

Interview Question 2

What is the scope of a CTE?

A CTE exists only for the statement immediately following its definition.

Example:

EXAMPLE / FLOW

```
WITH GiftCTE AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift
)

SELECT *
FROM GiftCTE;
```

After this statement finishes:

EXAMPLE / FLOW

```
SELECT *
FROM GiftCTE;
```

does not work.

Easy answer:

A CTE has one-statement scope.

Interview Question 3

What is the difference between a CTE and a temporary table?

A CTE is a named query expression used by one statement.

A temp table is a temporary database object that can be reused by several statements and can have indexes.

Example CTE:

EXAMPLE / FLOW

```
WITH Giving AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS TotalGiving
    FROM fundraising.Gift
    GROUP BY ConstituentId
)

SELECT *
FROM Giving;
```

Temp table equivalent:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    SUM(TotalAmount) AS TotalGiving
INTO #Giving
FROM fundraising.Gift
GROUP BY ConstituentId;
```

Use the temp table when intermediate data needs to be materialized and reused.

Interview Question 4

Does a CTE improve performance automatically?

No.

A CTE mainly improves query structure.

Performance still depends on:

EXAMPLE / FLOW

```
indexes

statistics

joins

filters

sorting

aggregation

window functions

row counts

execution plan
```

A CTE result is not automatically materialized for repeated reuse.

Interview Question 5 - Query Problem

Find each Constituent's latest Gift using a CTE.

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY GiftDate DESC,
                     GiftId DESC
        ) AS RowNumber

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM RankedGifts

WHERE RowNumber = 1;
```

Explanation

`PARTITION BY ConstituentId` restarts numbering for each donor.

`ORDER BY GiftDate DESC` puts the newest Gift first.

`GiftId DESC` handles ties.

The newest Gift gets:

EXAMPLE / FLOW

```
RowNumber = 1
```

and the outer query keeps it.

Interview Question 6 - Query Problem

Find the three largest Gifts made by each Constituent.

EXAMPLE / FLOW

```
WITH RankedGifts AS
(
    SELECT
        GiftId,
        ConstituentId,
        TotalAmount,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY TotalAmount DESC,
                     GiftId
        ) AS GiftPosition

    FROM fundraising.Gift
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT *
FROM RankedGifts

WHERE GiftPosition <= 3;
```

Explanation

The ranking resets per Constituent.

Therefore this means:

EXAMPLE / FLOW

```
Top 3 per donor
```

not:

EXAMPLE / FLOW

```
Top 3 across the whole Gift table.
```

Interview Question 7

What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

Given:

EXAMPLE / FLOW

```
1000
500
500
100
```

ROW_NUMBER:

EXAMPLE / FLOW

```
1,2,3,4
```

RANK:

EXAMPLE / FLOW

```
1,2,2,4
```

DENSE_RANK:

EXAMPLE / FLOW

```
1,2,2,3
```

Use **ROW_NUMBER** when exactly one ordered position is needed.

Use **RANK** for competition-style ranking.

Use **DENSE_RANK** when ties share rank without gaps.

Interview Question 8 - Query Problem

Rank Constituents by lifetime giving.

EXAMPLE / FLOW

```
WITH ConstituentGiving AS
(
    SELECT
        ConstituentId,
        SUM(TotalAmount) AS LifetimeGiving

    FROM fundraising.Gift

    GROUP BY ConstituentId
),
RankedGiving AS
(
    SELECT
        ConstituentId,
        LifetimeGiving,
```

EXAMPLE / FLOW (CONT. 2)

```
        DENSE_RANK()
        OVER
        (
            ORDER BY LifetimeGiving DESC
        ) AS GivingRank

    FROM ConstituentGiving
)

SELECT *
FROM RankedGiving;
```

Why Two CTE Stages?

Stage 1 calculates:

EXAMPLE / FLOW

```
one total per Constituent.
```

Stage 2 ranks those totals.

The separation makes the SQL easier to understand.

Interview Question 9

Can multiple CTEs exist in one query?

Yes.

Use one `WITH`:

EXAMPLE / FLOW

```
WITH FirstCTE AS
(
    ...
),
SecondCTE AS
(
    ...
)

SELECT ...
```

A later CTE may reference one defined before it.

Example:

EXAMPLE / FLOW

```
GiftTotals
   ↓
RankedGiftTotals
```

Interview Question 10

Can ORDER BY be used inside a CTE?

Ordinary final-result `ORDER BY` is normally not allowed directly inside the CTE definition except in permitted contexts such as when associated with `TOP` or `OFFSET/FETCH`.

Normally:

EXAMPLE / FLOW

```
WITH GiftCTE AS
(
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift
)

SELECT *
FROM GiftCTE

ORDER BY TotalAmount DESC;
```

Window-function `ORDER BY` is different and is allowed inside `OVER(...)`.

Interview Question 11 - Query Problem

Calculate total allocated amount per Fund.

EXAMPLE / FLOW

```
WITH FundAllocationTotals AS
(
    SELECT
        FundId,
        COUNT(DISTINCT GiftId) AS GiftCount,
        SUM(AllocationAmount) AS TotalAllocated

    FROM fundraising.GiftAllocation

    GROUP BY FundId
)

SELECT
    f.FundId,
    f.FundName,
    a.GiftCount,
    a.TotalAllocated

FROM FundAllocationTotals AS a
```

EXAMPLE / FLOW (CONT. 2)

```
INNER JOIN fundraising.Fund AS f
    ON f.FundId = a.FundId;
```

Explanation

The CTE aggregates the large child table first.

Then the final query adds Fund information.

Interview Question 12 - Query Problem

Find Gifts whose allocations do not equal Gift.TotalAmount.

EXAMPLE / FLOW

```
WITH AllocationTotals AS
(
    SELECT
        GiftId,
        SUM(AllocationAmount) AS AllocationTotal

    FROM fundraising.GiftAllocation

    GROUP BY GiftId
)

SELECT
    g.GiftId,
    g.TotalAmount,
    a.AllocationTotal,
    g.TotalAmount - a.AllocationTotal AS Difference

FROM fundraising.Gift AS g
```

EXAMPLE / FLOW (CONT. 2)

```
INNER JOIN AllocationTotals AS a
    ON a.GiftId = g.GiftId

WHERE g.TotalAmount <> a.AllocationTotal;
```

Explanation

First calculate allocation total.

Then compare:

EXAMPLE / FLOW

```
Gift.TotalAmount
vs
SUM(GiftAllocation.AllocationAmount)
```

This is a strong example of using a CTE for business validation.

Interview Question 13

What is a recursive CTE?

A recursive CTE references itself.

It contains:

EXAMPLE / FLOW

```
Anchor member

UNION ALL

Recursive member
```

The anchor creates the starting rows.

The recursive member repeatedly creates the next set of rows.

SQL Server stops when the recursive member returns no rows.

Interview Question 14 - Query Problem

Generate every month in a Campaign using a recursive CTE.

EXAMPLE / FLOW

```
WITH CampaignMonths AS
(
    SELECT
        CampaignId,

        CAST
        (
            DATEFROMPARTS
            (
                YEAR(StartDate),
                MONTH(StartDate),
                1
            )
            AS DATE
        ) AS MonthStart,

        EndDate

    FROM fundraising.Campaign
```

EXAMPLE / FLOW (CONT. 2)

```
    WHERE CampaignId = 200
      AND EndDate IS NOT NULL

    UNION ALL

    SELECT
        CampaignId,

        CAST
        (
            DATEADD
            (
                MONTH,
                1,
                MonthStart
            )
            AS DATE
        ),
```

EXAMPLE / FLOW (CONT. 3)

```
        EndDate

    FROM CampaignMonths

    WHERE DATEADD
          (
              MONTH,
              1,
              MonthStart
          )
          <=
          DATEFROMPARTS
          (
              YEAR(EndDate),
              MONTH(EndDate),
              1
          )
)
```

EXAMPLE / FLOW (CONT. 4)

```
SELECT
    CampaignId,
    MonthStart

FROM CampaignMonths

OPTION
(
    MAXRECURSION 100
);
```

Explanation

Anchor:

EXAMPLE / FLOW

```
first Campaign month
```

Recursive member:

EXAMPLE / FLOW

```
previous month + 1 month
```

Termination:

EXAMPLE / FLOW

```
stop after Campaign EndDate.
```

Interview Question 15 — What is MAXRECURSION?

`MAXRECURSION` controls how many recursion levels SQL Server allows.

Default:

EXAMPLE / FLOW

```
100
```

Allowed explicit values:

EXAMPLE / FLOW

```
0 through 32767
```

`0` means unlimited recursion.

Use it to protect recursive queries, not to hide incorrect recursive logic.

Interview Question 16 — What is the difference between CTE and subquery?

Both can express intermediate query logic.

CTE:

EXAMPLE / FLOW

```
named
often easier for multi-step logic
supports recursion
```

Subquery:

EXAMPLE / FLOW

```
inline
often simpler for small logic
```

They are not automatically different in performance merely because one uses `WITH`.

The optimizer determines the execution plan.

Interview Question 17 - Query Problem — Find the latest Interaction for each Constituent.

EXAMPLE / FLOW

```
WITH RankedInteractions AS
(
    SELECT
        InteractionId,
        ConstituentId,
        FundraiserUserId,
        InteractionType,
        InteractionDate,

        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY InteractionDate DESC,
                     InteractionId DESC
        ) AS RowNumber

    FROM engagement.Interaction
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT *
FROM RankedInteractions
WHERE RowNumber = 1;
```

Explanation

The CTE makes a common CRM/fundraising requirement simple:



Give me the latest staff interaction with each donor/prospect.

Interview Question 18 - Query Problem

Count incomplete Tasks assigned to every AppUser.

EXAMPLE / FLOW

```
WITH OpenTasks AS
(
    SELECT
        AssignedToUserId,
        COUNT(*) AS OpenTaskCount

    FROM engagement.Task

    WHERE CompletedAtUtc IS NULL

    GROUP BY AssignedToUserId
)

SELECT
    u.AppUserId,
    u.DisplayName,
    o.OpenTaskCount

FROM OpenTasks AS o
```

EXAMPLE / FLOW (CONT. 2)

```
INNER JOIN core.AppUser AS u
    ON u.AppUserId = o.AssignedToUserId

ORDER BY
    o.OpenTaskCount DESC;
```

Explanation

The CTE performs the aggregate at the Task level.

Then the outer query retrieves AppUser details.

Interview Question 19

Can a CTE be updated?

Yes, when SQL Server can map the CTE rows to updateable underlying table rows.

Example:

EXAMPLE / FLOW

```
WITH OverdueTasks AS
(
    SELECT
        TaskId,
        Priority,
        DueDate,
        CompletedAtUtc

    FROM engagement.Task

    WHERE DueDate < CAST(SYSUTCDATETIME() AS DATE)
      AND CompletedAtUtc IS NULL
)

UPDATE OverdueTasks
SET Priority = 'High';
```

But an aggregated CTE such as:

EXAMPLE / FLOW

```
one row containing SUM(TotalAmount)
```

does not correspond to one simple Gift row and generally cannot be updated that way.

Interview Question 20 - Full Problem

Requirement

Create a report showing:

EXAMPLE / FLOW

```
Constituent name

Gift count

Lifetime giving

Largest Gift

Latest Gift date

Lifetime giving rank

Latest Interaction date
```

Solution

EXAMPLE / FLOW

```
WITH GivingSummary AS
(
    SELECT
        ConstituentId,
        COUNT(*) AS GiftCount,
        SUM(TotalAmount) AS LifetimeGiving,
        MAX(TotalAmount) AS LargestGift,
        MAX(GiftDate) AS LatestGiftDate

    FROM fundraising.Gift

    GROUP BY ConstituentId
),
RankedGiving AS
(
    SELECT
        ConstituentId,
        GiftCount,
        LifetimeGiving,
        LargestGift,
```

EXAMPLE / FLOW (CONT. 2)

```
        LatestGiftDate,

        DENSE_RANK()
        OVER
        (
            ORDER BY LifetimeGiving DESC
        ) AS GivingRank

    FROM GivingSummary
),
RankedInteractions AS
(
    SELECT
        ConstituentId,
        InteractionDate,
```

EXAMPLE / FLOW (CONT. 3)

```
        ROW_NUMBER()
        OVER
        (
            PARTITION BY ConstituentId
            ORDER BY InteractionDate DESC,
                     InteractionId DESC
        ) AS InteractionRank

    FROM engagement.Interaction
),
LatestInteraction AS
(
    SELECT
        ConstituentId,
        InteractionDate

    FROM RankedInteractions
```

EXAMPLE / FLOW (CONT. 4)

```
    WHERE InteractionRank = 1
)

SELECT
    c.ConstituentId,
    c.DisplayName,

    rg.GiftCount,
    rg.LifetimeGiving,
    rg.LargestGift,
    rg.LatestGiftDate,
    rg.GivingRank,

    li.InteractionDate
        AS LatestInteractionDate

FROM RankedGiving AS rg
```

EXAMPLE / FLOW (CONT. 5)

```
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = rg.ConstituentId

LEFT JOIN LatestInteraction AS li
    ON li.ConstituentId = rg.ConstituentId

ORDER BY
    rg.GivingRank,
    c.ConstituentId;
```

Understand the Solution

First CTE:

EXAMPLE / FLOW

```
GivingSummary
→ One giving summary per Constituent.
```

Second CTE:

EXAMPLE / FLOW

```
RankedGiving
→ Rank Constituents by lifetime giving.
```

Third CTE:

EXAMPLE / FLOW

```
RankedInteractions
→ Number Interactions newest-to-oldest per Constituent.
```

Fourth CTE:

EXAMPLE / FLOW

```
LatestInteraction
→ Keep only InteractionRank = 1.
```

Final query:

EXAMPLE / FLOW

```
Join everything to Constituent.
```

This demonstrates why CTEs are valuable.

Without logical names, the same query could become a deeply nested and much harder-to-read statement.

Final Revision Map

EXAMPLE / FLOW

```
RANKING FUNCTIONS
        |
        +---- ROW_NUMBER
        |     Every row unique
        |
        +---- RANK
        |     Tie + gap
        |
        +---- DENSE_RANK
        |     Tie + no gap
        |
        +---- NTILE
              Ordered groups
```

EXAMPLE / FLOW

```
NORMAL CTE

Base table
   |
   v
CTE query
   |
   v
Named logical result
   |
   v
SELECT / UPDATE / DELETE
```

EXAMPLE / FLOW

```
MULTIPLE CTEs

Gift
  |
  v
GivingSummary
  |
  v
RankedGiving
  |
  v
Final report
```

EXAMPLE / FLOW

```
RECURSIVE CTE

Anchor
  |
  v
Recursive member
  |
  v
Next result
  |
  v
Recursive member again
  |
  v
No rows
  |
  v
STOP
```

EXAMPLE / FLOW

```
CHOOSING THE RIGHT TOOL

Simple query
→ SELECT

Small inline intermediate query
→ Subquery

Complex one-statement logical stages
→ CTE

Hierarchy / repeated sequence
→ Recursive CTE

Intermediate data reused across statements
→ #Temp table

Persistent reusable database query
→ View
```

Final Easy Memory Rule

A CTE is a temporary name for query logic inside one SQL statement. Use it when naming logical steps makes SQL easier to understand, especially for aggregation, ranking, Top-N-per-group, latest-row queries, and recursion.

For your `FundraisingDb`, remember these practical patterns:

EXAMPLE / FLOW

```
Latest Gift per Constituent
→ CTE + ROW_NUMBER

Top Gifts per Constituent
→ CTE + ROW_NUMBER

Rank donors by lifetime giving
→ GROUP BY CTE + DENSE_RANK

Rank campaigns
→ Aggregate CTE + RANK

Fund totals
→ GiftAllocation GROUP BY CTE

Latest donor Interaction
→ CTE + ROW_NUMBER
```

EXAMPLE / FLOW (CONT. 2)

```
Open Tasks by AppUser
→ Aggregate CTE

Campaign month sequence
→ Recursive CTE
```

And the most important performance rule is:

Do not use a CTE because you expect it to make SQL faster. Use it because it expresses the query clearly. Then evaluate performance using the actual execution plan, indexes, statistics, row counts, and workload.